

Machine Learning Programming Exercise 2: Logistic Regression

adapted to Python language from Coursera/Andrew Ng

September 24, 2024

1 Introduction

In this exercise, you will implement logistic regression and get to see how it works on data. To get started with the exercise, you will need to download the starter code and unzip its contents to the directory where you wish to complete the exercise. To complement the lesson some video lectures are available on YouTube. Links are provided in this document.

2 Files included in this exercise

- **ex2.py** - Python script that will help step you through the exercise
- **ex2_reg.py** - Python script for the second part of the exercise
- **ex2_multiclass.py** - Python script for the later parts of the exercise
- **ex2data1.txt** - Dataset for linear regression with one variable
- **ex2data2.txt** - Dataset for linear regression with multiple variables
- **ex2data3.txt** - Dataset for linear regression with multiple variables and multiple classes
- **plotDecisionBoundary.txt** - Plots the data points X and y into a new figure with the decision boundary defined by theta
- ★ **plotData.py** - function to display the dataset
- ★ **sigmoid.py** -Sigmoid Function
- ★ **gradientDescent.py** - Function to run gradient descent
- ★ **costFunction.py** - Logistic Regression Cost Function
- ★ **gradientFunction.py** - Logistic Regression Gradient Function
- ★ **costFunctionReg.py** - Regularized Logistic Regression Cost variables
- ★ **gradientFunctionReg.py** - Regularized Logistic Regression Gradient Function
- ★ **predict.py** - Logistic Regression Prediction Function
- ★ **learnOneVsAll.py** - Train a one-vs-all multi-class classifier
- ★ **predictOneVsAll.py** - Predict using a one-vs-all multi-class classifier

[★] indicates files you will need to complete

Throughout the exercise, you will be using the scripts **ex2.py**, **ex2_reg.py** and **ex2_multiclass.py**. These scripts set up the dataset for the problems and make calls to functions that you will write. You do not need to modify either of them. You are only required to modify functions in other files, by following the instructions in this assignment.

3 Logistic Regression

You may wish to supplement the lesson by watching this video: [this video](#)!

In this part of the exercise, you will build a logistic regression model to predict whether a student gets admitted into a university. Suppose that you are the administrator of a university department and you want to determine each applicant's chance of admission based on their results on two exams. You have historical data from previous applicants that you can use as a training set for logistic regression. For each training example, you have the applicant's scores on two exams and the admissions decision. Your task is to build a classification model that estimates an applicant's probability of admission based on the scores from those two exams. This outline and the framework code in **ex2.py** will guide you through the exercise.

3.1 Visualizing the data

Before starting to implement any learning algorithm, it is always good to visualize the data if possible. In the first part of **ex2.py**, the code will load the data and display it on a 2-dimensional plot by calling the function **plotData**. You will now complete the code in **plotData** so that it displays a figure like Figure 1, where the axes are the two exam scores, and the positive and negative examples are shown with different markers.

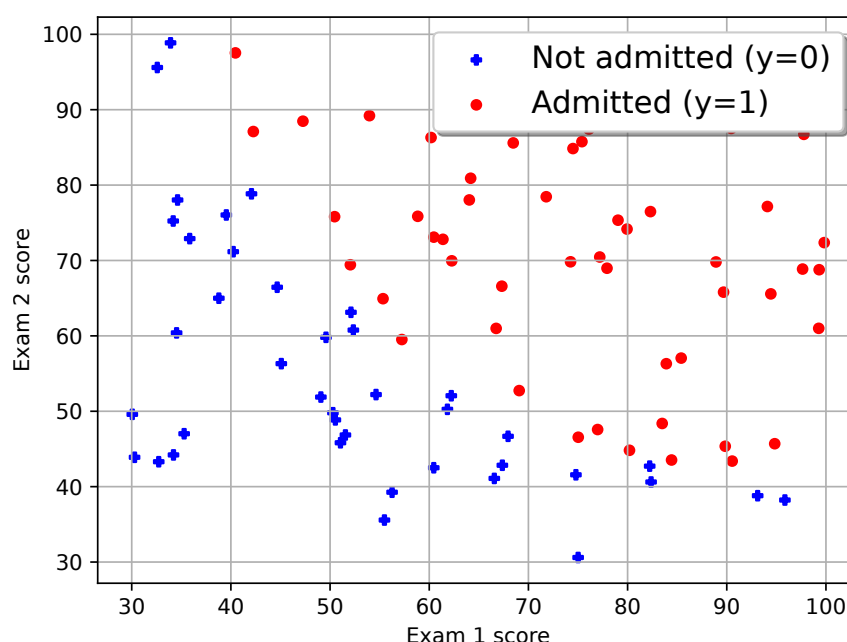


Figure 1: Scatter plot of training data

We have left **plotData.py** empty so you can try to implement it yourself, or you can copy the following code or refer to it.

```
pos = X[(y==1).flatten(),:]
neg = X[(y==0).flatten(),:]

plt.figure()
plt.scatter(neg[:, 0], neg[:, 1], s=20, marker='x', color=cmap(0))
plt.scatter(pos[:, 0], pos[:, 1], s=20, marker='o', color=cmap(cmap.N))
plt.xlabel('Exam 1 score')
plt.ylabel('Exam 2 score')
plt.grid(True)
plt.legend(['Not admitted (y=0)', 'Admitted (y=1)'], loc='upper right', shadow=True,
           ↪ fontsize='x-large', numpoints=1)
```

3.2 Implementation

You may wish to supplement the lesson by watching this video: [Hypothesis representation](#), [Decision boundary](#).

3.2.1 Sigmoid function

Before you start with the actual cost function, recall that the logistic regression hypothesis is defined as:

$$h_{\theta}(\mathbf{x}) = g(\mathbf{x}^T \theta),$$

where function g is the sigmoid function. The sigmoid function is defined as:

$$g(z) = \frac{1}{1 + e^{-z}}.$$

Your first step is to implement this function in **sigmoid.py** so it can be called by the rest of your program. When you are finished, try testing a few values by calling **sigmoid(x)** at the python command line. For large positive values of $\mathbf{x}$, the sigmoid should be close to 1, while for large negative values, the sigmoid should be close to 0. Evaluating **sigmoid(0)** should give you exactly 0.5. Your code should also work with vectors and matrices. For a vector or a matrix, your function should perform the sigmoid function on every element.

3.2.2 Cost function and gradient

You may wish to supplement the lesson by watching this video: [Cost function](#), [Simplified cost function and gradient descent](#).

Now you will implement the cost function and gradient for logistic regression. Complete the code in **costFunction.py** and **gradientFunction.py** to return the cost and gradient. Recall that the cost function in logistic regression is

$$J(\theta) = \frac{1}{m} \sum_{i=0}^{m-1} [-y^i \log(h_{\theta}(\mathbf{x}^i)) - (1 - y^i) \log(1 - h_{\theta}(\mathbf{x}^i))],$$

and the gradient of the cost is a vector where the j^{th} element (for $j = 0, 1, 2$) is defined as follows:

$$\nabla_{\theta} J(\theta) = \begin{pmatrix} \frac{\partial}{\partial \theta_0} J(\theta) \\ \frac{\partial}{\partial \theta_1} J(\theta) \\ \frac{\partial}{\partial \theta_2} J(\theta) \end{pmatrix} = \begin{pmatrix} \frac{1}{m} \sum_{i=0}^{m-1} [(h_{\theta}(\mathbf{x}^i) - y^i)] \\ \frac{1}{m} \sum_{i=0}^{m-1} [(h_{\theta}(\mathbf{x}^i) - y^i) x_1^i] \\ \frac{1}{m} \sum_{i=0}^{m-1} [(h_{\theta}(\mathbf{x}^i) - y^i) x_2^i] \end{pmatrix}$$

Note that while this gradient looks identical to the linear regression gradient, the formula is actually different because linear and logistic regression have different definitions of $h_{\theta}(\mathbf{x})$.

Once you are done, **ex2.py** will call your **costFunction** using the initial parameters of θ . You should see that the cost is about 0.693.

3.2.3 Learning parameters using **fmin_tnc**

In the previous assignment, you found the optimal parameters of a linear regression model by implementing gradient descent. You wrote a cost function and calculated its gradient, then took a gradient descent step accordingly.

This time, instead of taking gradient descent steps, you will use a python built-in function called **fmin_tnc**. Python's **fmin_tnc** is an optimization solver that finds the minimum of an unconstrained¹ function. For logistic regression, you want to optimize the cost function $J(\theta)$ with parameters θ .

Concretely, you are going to use **fmin_tnc** to find the best parameters θ for the logistic regression cost function, given a fixed dataset (of $\mathbf{X}$ and $\mathbf{y}$ values). You will pass to **fmin_tnc** the following inputs:

¹Constraints in optimization often refer to constraints on the parameters, for example, constraints that bound the possible values θ can take (e.g., $\theta \leq 1$). Logistic regression does not have such constraints since θ is allowed to take any real value.

- A function that, when given the training set and a particular θ , computes the logistic regression cost for the dataset $(\mathbf{X}, \mathbf{y})$
- The initial values of the parameters we are trying to optimize.
- A function that, when given the training set and a particular θ , computes the logistic regression gradient with respect to θ for the dataset $(\mathbf{X}, \mathbf{y})$

In **ex2.py**, we already have code written to call **fmin_tnc** with the correct arguments.

```
theta = opt.fmin_tnc(costFunction, initial_theta, gradientFunction, args=(X, y))
```

If you have completed the **costFunction** correctly, **fmin_tnc** will converge on the right optimization parameters and returns the final values of θ . Notice that by using **fmin_tnc**, you did not have to write any loops yourself, or set a learning rate like you did for gradient descent. This is all done by **fmin_tnc**: you only need to provide a function calculating the cost and the gradient.

Once **fmin_tnc** completes, **ex2.py** will call your **costFunction** function using the optimal parameters of θ . You should see that the cost is about 0.203.

This final θ value will then be used to plot the decision boundary on the training data, resulting in a figure similar to Figure 2. We also encourage you to look at the code in **plotDecisionBoundary.py** to see how to plot such a boundary using the θ values.

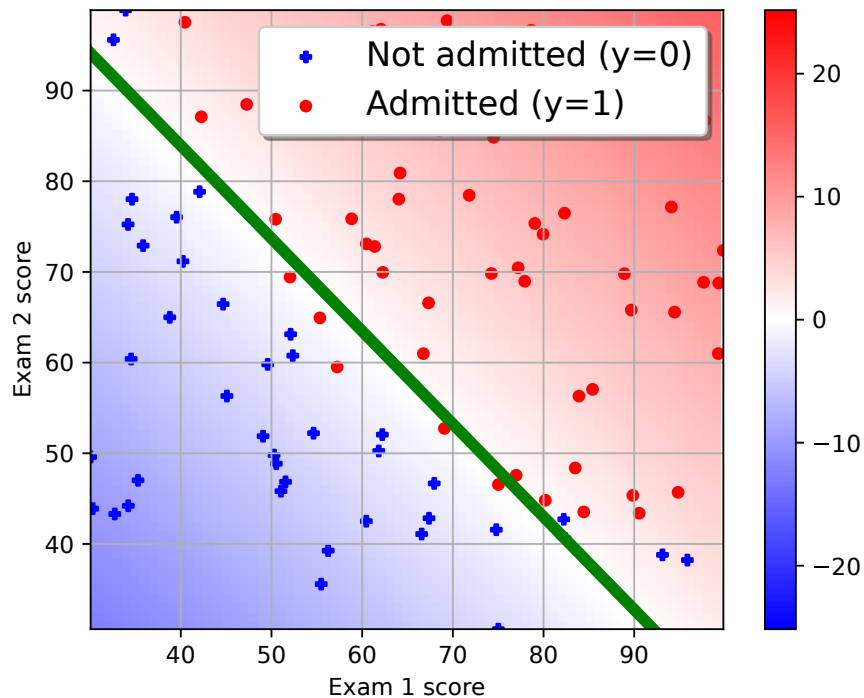


Figure 2: Training data with decision boundary.

3.2.4 Evaluating logistic regression

You may wish to supplement the lesson by watching this video: [The problem of overfitting](#).

After learning the parameters, you can use the model to predict whether a particular student will be admitted. For a student with an Exam 1 score of 45 and an Exam 2 score of 85, you should expect to see an admission probability of 0.774.

Another way to evaluate the quality of the parameters we have found is to see how well the learned model predicts on our training set. In this part, your task is to complete the code in **predict.py**. The predict function will produce 1 or 0 predictions given a dataset and a learned parameter vector θ .

After you have completed the code **predict.py**, the **ex2.py** script will proceed to report the training accuracy of your classifier by computing the percentage of examples it got correct.

4 Regularized logistic regression

You may wish to supplement the lesson by watching this video: [Regularized Logistic Regression, Regularized Logistic Regression](#).

In this part of the exercise, you will implement regularized logistic regression to predict whether microchips from a fabrication plant passes quality assurance (QA). During QA, each microchip goes through various tests to ensure it is functioning correctly.

Suppose you are the product manager of the factory and you have the test results for some microchips on two different tests. From these two tests, you would like to determine whether the microchips should be accepted or rejected. To help you make the decision, you have a dataset of test results on past microchips, from which you can build a logistic regression model.

You will use another script, `ex2_reg.py` to complete this portion of the exercise.

4.1 Visualizing the data

Similar to the previous parts of this exercise, `plotData` is used to generate a figure like Figure 3, where the axes are the two test scores, and the positive ($y = 1$, accepted) and negative ($y = 0$, rejected) examples are shown with different markers.

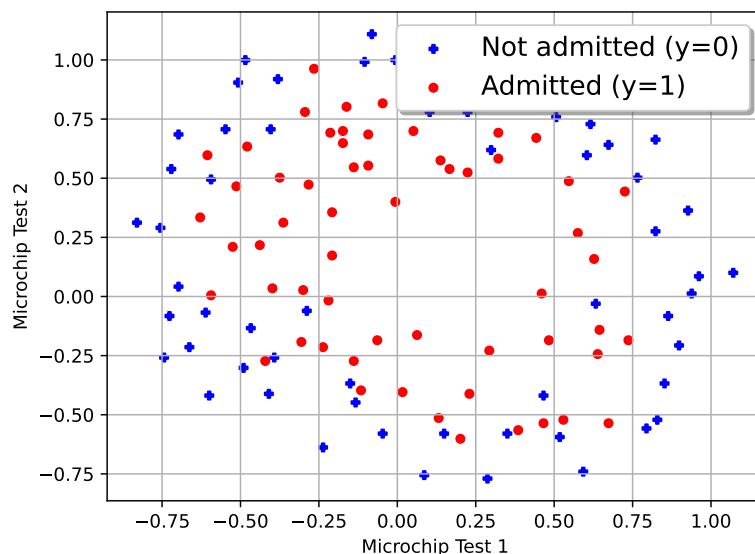


Figure 3: Plot of training data.

Figure 3 shows that our dataset cannot be separated into positive and negative examples by a straight-line through the plot. Therefore, a straight-forward application of logistic regression will not perform well on this dataset since logistic regression will only be able to find a linear decision boundary.

4.2 Feature mapping

One way to fit the data better is to create more features from each data point. In the provided function `mapFeature` in `plotDecisionBoundary.py`, we will map the features into all polynomial terms of x_1 and x_2 up to the sixth power.

$$\text{mapFeature}(\mathbf{x}) = \begin{pmatrix} 1 \\ x_1 \\ x_2 \\ x_1^2 \\ x_1x_2 \\ x_2^2 \\ x_1^3 \\ \vdots \\ x_1x_2^5 \\ x_2^6 \end{pmatrix}$$

As a result of this mapping, our vector of two features (the scores on two QA tests) has been transformed into a 28-dimensional vector. A logistic regression classifier trained on this higher-dimension feature vector will have a more complex decision boundary and will appear nonlinear when drawn in our 2-dimensional plot.

While the feature mapping allows us to build a classifier with more complex decision boundaries, it is also more susceptible to overfitting. In the next parts of the exercise, you will implement regularized logistic regression to fit the data and also see for yourself how regularization can help combat the overfitting problem.

4.3 Cost function and gradient

Now you will implement code to compute the cost function and gradient for regularized logistic regression. Complete the code in **costFunctionReg.py** and **gradientFunctionReg.py** to return the cost and gradient.

Recall that the regularized cost function in logistic regression is

$$J(\boldsymbol{\theta}) = \frac{1}{m} \sum_{i=0}^{m-1} [-y^i \log(h_{\boldsymbol{\theta}}(\mathbf{x}^i)) - (1 - y^i) \log(1 - h_{\boldsymbol{\theta}}(\mathbf{x}^i))] + \frac{\lambda}{2m} \sum_{j=1}^n \theta_j^2$$

Implementation note: Note that you should not regularize the parameter θ_0 . Thus, the final summation above is for $j = 1$ to n , not $j = 0$ to n .

The gradient of the cost function is a vector where the j -th element is defined as follows:

$$\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}) = \begin{pmatrix} \frac{\partial}{\partial \theta_0} J(\boldsymbol{\theta}) \\ \frac{\partial}{\partial \theta_1} J(\boldsymbol{\theta}) \\ \vdots \\ \frac{\partial}{\partial \theta_n} J(\boldsymbol{\theta}) \end{pmatrix} = \begin{pmatrix} \frac{1}{m} \sum_{i=0}^{m-1} [(h_{\boldsymbol{\theta}}(\mathbf{x}^i) - y^i) x_0^i] \\ \frac{1}{m} \sum_{i=0}^{m-1} [(h_{\boldsymbol{\theta}}(\mathbf{x}^i) - y^i) x_1^i] + \frac{\lambda}{m} \theta_1 \\ \vdots \\ \frac{1}{m} \sum_{i=0}^{m-1} [(h_{\boldsymbol{\theta}}(\mathbf{x}^i) - y^i) x_n^i] + \frac{\lambda}{m} \theta_n \end{pmatrix}$$

Once you are done, **ex2_reg.py** will call your **costFunctionReg** function using the initial value of $\boldsymbol{\theta}$ (initialized to all zeros). You should see that the cost is about 0.693.

4.3.1 Learning parameters using **fmin_tnc**

Similar to the previous parts, you will use **fmin_tnc** to learn the optimal parameters $\boldsymbol{\theta}$. If you have completed the cost and gradient for regularized logistic regression (**costFunctionReg.py** and **gradientFunctionReg.py**) correctly, you should be able to step through the next part of **ex2_reg.py** to learn the parameters $\boldsymbol{\theta}$ using **fmin_tnc**.

4.4 Plotting the decision boundary

To help you visualize the model learned by this classifier, we have provided the function **plotDecisionBoundary.py** which plots the (non-linear) decision boundary that separates the positive and negative examples. In **plotDecisionBoundary.py**, we plot the non-linear decision boundary by computing the classifier's predictions on an evenly spaced grid and then draw a contour plot of where the predictions change from $y = 0$ to $y = 1$.

After learning the parameters θ , the next step in **ex2_reg.py** will plot a decision boundary similar to Figure 5.

4.5 Impact of λ

In this part of the exercise, you will get to try out different regularization parameters for the dataset to understand how regularization prevents overfitting.

Notice the changes in the decision boundary as you vary λ . With a small λ , you should find that the classifier gets almost every training example correct, but draws a very complicated boundary, thus overfitting the data (Figure 4). This is not a good decision boundary: for example, it predicts that a point at $\mathbf{x} = (-0.25; 1.5)$ is accepted ($y = 1$), which seems to be an incorrect decision given the training set.

With a larger λ , you should see a plot that shows an simpler decision boundary which still separates the positives and negatives fairly well. However, if λ is set to too high a value, you will not get a good fit and the decision boundary will not follow the data so well, thus underfitting the data (Figure 6).

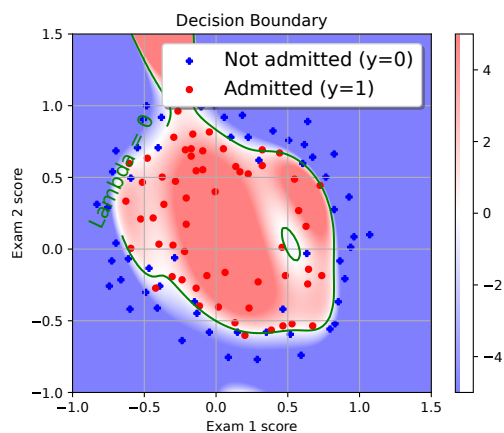


Figure 4: No regularization (Overfitting) ($\lambda = 0$)

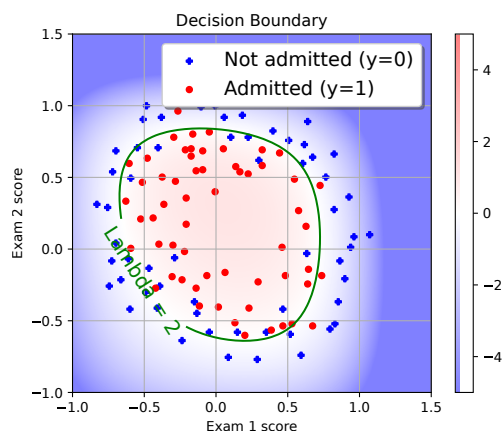


Figure 5: Training data with decision boundary ($\lambda = 2$)

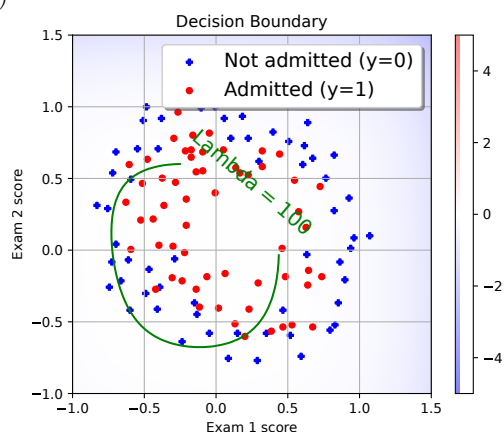


Figure 6: Too much regularization (Underfitting) ($\lambda = 100$)

5 Multi-class classification

In this last part of the assignment, you will implement one-vs-all regularized logistic regression to recognize hand-written digits.

You may wish to supplement the lesson by watching this video: [Multi-class classification One Vs All](#).

Throughout the exercise, you will be using the scripts `ex2_multiclass.py`. These scripts set up the dataset for the problems and make calls to functions that you will write. You do not need to modify these scripts. You are only required to modify functions in other files, by following the instructions in this assignment.

For this exercise, you will use regularized logistic regression to recognize handwritten digits (from 0 to 9). Automated handwritten digit recognition is widely used today - from recognizing zip codes (postal codes) on mail envelopes to recognizing amounts written on bank checks. This exercise will show you how the methods you've learned in the first parts can be used for this classification task. You will extend your previous implementation of logistic regression and apply it to one-vs-all classification.

5.1 Dataset

You are given a data set in `ex2data3.mat` that contains 5000 training examples of handwritten digits. Two matrices can be read directly into your program by using the command:

```
2  # Load saved matrices from file
   data = np.load('ex2data3.npz')
   X, y = data['X'], data['y']
4  m,n = X.shape
   # X is an array (5000 x 400)
6  # y is an array (5000 x 1)
```

The matrices X and y will now be in your Python environment. There are 5000 training examples in `ex2data3.mat`, where each training example is a 20 pixel by 20 pixel grayscale image of the digit. Each pixel is represented by a floating point number indicating the grayscale intensity at that location. The 20 by 20 grid of pixels is “unrolled” into a 400-dimensional vector. Each of these training examples become a single row in our data matrix X . This gives us a $m = 5000$ by $n = 400$ matrix X where every row is a training example for a handwritten digit image.

$$X = \begin{bmatrix} (\mathbf{x}^0)^T \\ (\mathbf{x}^1)^T \\ \vdots \\ (\mathbf{x}^{m-1})^T \end{bmatrix}$$

where $\mathbf{x}_{i=0,\dots,m-1}^i$ is a n -dimensional vector. The second part of the training set is a 5000-dimensional vector $\mathbf{y}$ that contains labels for the training set.

5.2 Visualizing the data

You will begin by visualizing a subset of the training set. In Part 1 of `ex2_multiclass.py`, the code randomly selects 100 rows from X and passes those rows to the `displayData` function. This function maps each row to a 20 pixel by 20 pixel grayscale image and displays the images together. We have provided the `displayData` function, and you are encouraged to examine the code to see how it works. After you run this step, you should see an image like Figure 7.



Figure 7: Examples from the dataset.

5.3 Vectorizing Logistic Regression

You will be using multiple one-vs-all logistic regression models to build a multi-class classifier. Since there are 10 classes, you will need to train 10 separate logistic regression classifiers. To make this training efficient, it is important to ensure that your code is well vectorized. In this section, you will implement a vectorized version of logistic regression that does not employ any for loops. You can use your code in the last exercise as a starting point for this exercise.

If in the previous parts, you have already computed a vectorized version (without any loops) of your regularized cost function and your regularized gradient function, you have nothing to do in your **costFunctionReg.py** and **gradientFunctionReg.py**. Otherwise, you will find details on how to vectorize codes in the sequel.

5.3.1 Vectorizing the cost function

We will begin by writing a vectorized version of the cost function. Recall that in regularized logistic regression, the cost function is:

$$J(\theta) = \frac{1}{m} \sum_{i=0}^{m-1} [-y^i \log(h_{\theta}(\mathbf{x}^i)) - (1 - y^i) \log(1 - h_{\theta}(\mathbf{x}^i))] + \frac{\lambda}{2m} \sum_{j=1}^n \theta_j^2$$

To compute each element in the summation, we have to compute $h_{\theta}(\mathbf{x}^i)$ for every example i , where $h_{\theta}(\mathbf{x}^i) = g((\mathbf{x}^i)^T \theta)$ and $g(z) = \frac{1}{1+e^{-z}}$ is the sigmoid function. It turns out that we can compute this quickly for all our examples by using matrix multiplication. Let us define $\mathbf{X}$ and θ as

$$\mathbf{X} = \begin{bmatrix} (\mathbf{x}^0)^T \\ (\mathbf{x}^1)^T \\ \vdots \\ (\mathbf{x}^{m-1})^T \end{bmatrix} \quad \text{and} \quad \theta = \begin{bmatrix} \theta_0 \\ \theta_1 \\ \vdots \\ \theta_n \end{bmatrix}$$

Then, by computing the matrix product $\mathbf{X}\theta$, we have

$$X\theta = \begin{bmatrix} (\mathbf{x}^0)^T \theta \\ (\mathbf{x}^1)^T \theta \\ \vdots \\ (\mathbf{x}^{m-1})^T \theta \end{bmatrix}$$

Your job is to write the vectorized regularized cost function in the file **costFunctionReg.py**. Your implementation should use the strategy we presented above to calculate $X\theta$. You should also use a vectorized approach for the rest of the cost function. A fully vectorized version of **costFunctionReg.py** should not contain any loops. Hint: You might want to use the element-wise multiplication operation (*) and the sum operation when writing this function. The matrix multiplication operation is either @ or **np.dot()**.

5.3.2 Vectorizing the gradient

Recall that the gradient of the regularized logistic regression cost is a vector $\nabla_{\theta} J(\theta)$ with all the partial derivatives of $J(\theta)$ for all θ_j :

$$\begin{aligned} \nabla_{\theta} J(\theta) &= \begin{pmatrix} \frac{\partial}{\partial \theta_0} J(\theta) \\ \frac{\partial}{\partial \theta_1} J(\theta) \\ \vdots \\ \frac{\partial}{\partial \theta_n} J(\theta) \end{pmatrix} = \begin{pmatrix} \frac{1}{m} \sum_{i=0}^{m-1} [(h_{\theta}(\mathbf{x}^i) - y^i) x_0^i] \\ \frac{1}{m} \sum_{i=0}^{m-1} [(h_{\theta}(\mathbf{x}^i) - y^i) x_1^i] + \frac{\lambda}{m} \theta_1 \\ \vdots \\ \frac{1}{m} \sum_{i=0}^{m-1} [(h_{\theta}(\mathbf{x}^i) - y^i) x_n^i] + \frac{\lambda}{m} \theta_n \end{pmatrix} \\ &= \frac{1}{m} \sum_{i=0}^{m-1} (h_{\theta}(\mathbf{x}^i) - y^i) \mathbf{x}^i + \frac{\lambda}{m} [0, \theta_1, \dots, \theta_n]^T \\ &= \frac{1}{m} \mathbf{X}^T (h_{\theta}(\mathbf{X}) - \mathbf{y}) + \frac{\lambda}{m} [0, \theta_1, \dots, \theta_n]^T \end{aligned} \tag{1}$$

$$\tag{2}$$

where

$$h_{\theta}(\mathbf{X}) - \mathbf{y} = \begin{bmatrix} h_{\theta}(\mathbf{x}^0) - y^0 \\ h_{\theta}(\mathbf{x}^1) - y^1 \\ \vdots \\ h_{\theta}(\mathbf{x}^{m-1}) - y^{m-1} \end{bmatrix}$$

Note that $\mathbf{x}^i$ is a vector, while $(h_{\theta}(\mathbf{x}^i) - y^i)$ is a scalar (single number). To understand the last step of the derivation, let $\beta_i = (h_{\theta}(\mathbf{x}^i) - y^i)$ and observe that:

$$\sum_i \beta_i \mathbf{x}^i = \begin{bmatrix} | & | & | & | \\ \mathbf{x}^1 & \mathbf{x}^2 & \dots & \mathbf{x}^m \\ | & | & | & | \end{bmatrix} \begin{bmatrix} \beta_0 \\ \beta_1 \\ \vdots \\ \beta_{m-1} \end{bmatrix} = X^T \beta$$

The expression above allows us to compute all the partial derivatives without any loops. If you are comfortable with linear algebra, we encourage you to work through the matrix multiplications above to convince yourself that the vectorized version does the same computations. You should now implement Equation 2 to compute the correct vectorized gradient. Once you are done, change the function **gradientFunctionReg.py** by implementing the gradient.

5.4 Learning a one-vs-all classifier

In this part of the exercise, you will discover the way to make an one-vs-all classification by training multiple regularized logistic regression classifiers, one for each of the K classes in our dataset (Figure 7). In the handwritten digits dataset, $K = 10$, but your code should work for any value of K .

You should now read carefully the code in **learnOneVsAll.py** to train one classifier for each class. In particular, note the code returns all the classifier parameters in a matrix $\Theta \in \mathbb{R}^{K \times (n+1)}$, where each row of Θ corresponds to the learned logistic regression parameters for one class. This is achieved with a “for”-loop from 0 to $K - 1$, training each classifier independently.

When training the classifier for class $k \in \{0, \dots, K - 1\}$, you will want a m -dimensional vector of labels $\mathbf{y}$, where $y_j \in \{0, 1\}$ indicates whether the j -th training instance belongs to class k ($y_j = 1$), or if it belongs to a different class ($y_j = 0$). You may note that logical operators is helpful for this task.

Python Tip : Logical arrays in Python are arrays which contain binary (true or false) elements. In Python, evaluating the expression $a == b$ for a vector a (of size $m \times 1$) and scalar b will return a vector of the same size as a with ones at positions where the elements of a are equal to b and zeroes where they are different. To see how this works for yourself, try the following code in Python:

```

2   a = np.arange(0,9,1) # Create a and b
   b = 3
4   p = (a == b) # You should try different values of b here
   print(p)
```

After you have carefully examined the code for **learnOneVsAll.py**, the script **ex2_multiclass.py** will continue to use your **learnOneVsAll** function to train a multiclass classifier.

5.5 One-vs-all prediction

After training your one-vs-all classifier, you can now use it to predict the digit contained in a given image. For each input, you should compute the “probability” that it belongs to each class using the trained logistic regression classifiers. Your one-vs-all prediction function will pick the class for which the corresponding logistic regression classifier outputs the highest probability and return the class label (0, 2,..., or $K - 1$) as the prediction for the input example.

You should now complete the code in **predictOneVsAll.py** to use the one-vs-all classifier to make predictions.

Once you are done, **ex2_multiclass.py** will call your **predictOneVsAll** function using the learned value of Θ . You should see that the training set accuracy is about 96.5% (i.e., it classifies 96.5% of the examples in the training set correctly).

5.6 Some questions, you have to answer...

Le codage n’est qu’un prétexte pour comprendre les différents concepts du machine learning. **Ici aucune équation...Exprimez avec vos mots les concepts pour les comprendre.**

1. Identifiez les différences d’approches entre le TP1 et le TP2 et les différences de résolution du problème.
2. A quoi sert le principe de régularisation ?
3. Décrivez le problème du sur-apprentissage et comment on y répond dans ce TP. Quelle est l’influence de la valeur de λ ?
4. Dans quelles circonstances et pour quelles raisons utilise-t-on l’approche multiclasse ? Quelles sont les différentes possibilités pour cette approche?